

Program 1

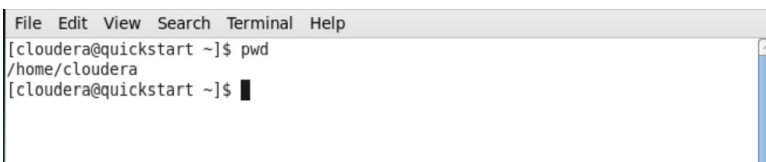
Install Hadoop and Implement the following file management tasks in Hadoop:

- Adding files and directories
- Retrieving files
- Deleting files and directories.
- **Hint: A typical Hadoop workflow creates data files (such as log files) elsewhere and copies them into HDFS using one of the above command line utilities**

Note: Every command for HDFS starts with **hdfs dfs**

Execute the following commands-

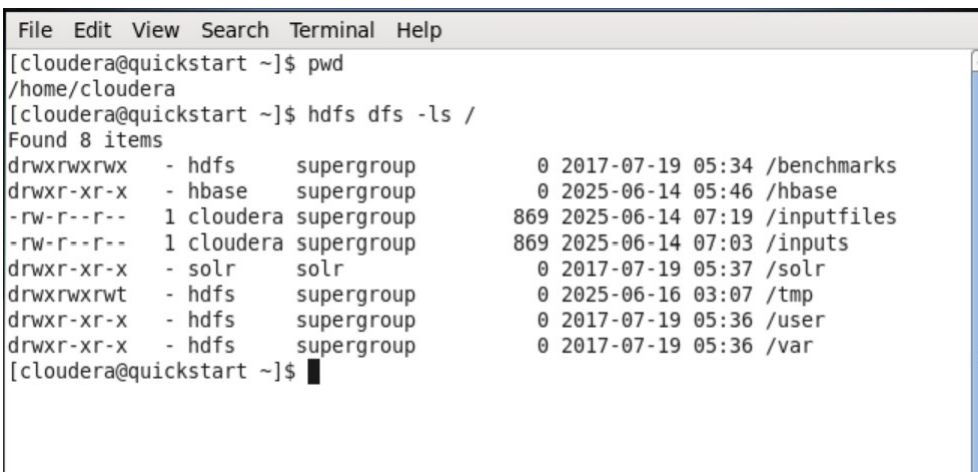
1. **pwd** - to know the present working directory of the local file system.



```
File Edit View Search Terminal Help
[cloudera@quickstart ~]$ pwd
/home/cloudera
[cloudera@quickstart ~]$
```

To explore the HDFS use following command

2. **hdfs dfs -ls /**



```
File Edit View Search Terminal Help
[cloudera@quickstart ~]$ pwd
/home/cloudera
[cloudera@quickstart ~]$ hdfs dfs -ls /
Found 8 items
drwxrwxrwx - hdfs supergroup 0 2017-07-19 05:34 /benchmarks
drwxr-xr-x - hbase supergroup 0 2025-06-14 05:46 /hbase
-rw-r--r-- 1 cloudera supergroup 869 2025-06-14 07:19 /inputfiles
-rw-r--r-- 1 cloudera supergroup 869 2025-06-14 07:03 /inputs
drwxr-xr-x - solr solr 0 2017-07-19 05:37 /solr
drwxrwxrwt - hdfs supergroup 0 2025-06-16 03:07 /tmp
drwxr-xr-x - hdfs supergroup 0 2017-07-19 05:36 /user
drwxr-xr-x - hdfs supergroup 0 2017-07-19 05:36 /var
[cloudera@quickstart ~]$
```

To explore the specific directory of HDFS for example user directory of HDFS

3. **hdfs dfs -ls /user**

```
[cloudera@quickstart ~]$ hdfs dfs -ls /user
Found 8 items
drwxr-xr-x - cloudera cloudera          0 2025-06-24 08:35 /user/cloudera
drwxr-xr-x - mapred  hadoop              0 2025-06-14 20:01 /user/history
drwxrwxrwx - hive    supergroup          0 2017-07-19 05:36 /user/hive
drwxrwxrwx - hue     supergroup          0 2025-06-24 08:40 /user/hue
drwxrwxrwx - jenkins supergroup          0 2017-07-19 05:35 /user/jenkins
drwxrwxrwx - oozie   supergroup          0 2017-07-19 05:35 /user/oozie
drwxrwxrwx - root    supergroup          0 2025-06-25 02:03 /user/root
drwxr-xr-x - hdfs    supergroup          0 2017-07-19 05:36 /user/spark
[cloudera@quickstart ~]$
```

To create the directory in the /user/root and list the /user/root use following command

4. hdfs dfs -mkdir /user/root/demodir

5. hdfs dfs -ls /user/root/

```
File Edit View Search Terminal Help
[cloudera@quickstart ~]$ hdfs dfs -mkdir /user/root/demodir
[cloudera@quickstart ~]$ hdfs dfs -ls /user/root
Found 15 items
drwx----- - root    supergroup          0 2025-06-15 02:32 /user/root/.staging
drwxr-xr-x - cloudera supergroup          0 2025-06-24 23:42 /user/root/citech
drwxr-xr-x - cloudera supergroup          0 2025-06-30 20:32 /user/root/demodir
drwxr-xr-x - cloudera supergroup          0 2025-06-16 01:43 /user/root/matin
drwxr-xr-x - cloudera supergroup          0 2025-06-16 01:44 /user/root/matout
drwxr-xr-x - cloudera supergroup          0 2025-06-16 23:03 /user/root/movie
drwxr-xr-x - cloudera supergroup          0 2025-06-16 23:31 /user/root/movietags
drwxr-xr-x - cloudera supergroup          0 2025-06-16 23:41 /user/root/movietags1
drwxr-xr-x - cloudera supergroup          0 2025-06-24 23:12 /user/root/myfloder
drwxr-xr-x - root    supergroup          0 2025-06-15 02:32 /user/root/output
drwxr-xr-x - cloudera supergroup          0 2025-06-25 02:03 /user/root/out
drwxr-xr-x - cloudera supergroup          0 2025-06-16 02:54 /user/root/pigdata
drwxr-xr-x - cloudera supergroup          0 2025-06-16 03:07 /user/root/pigoutput
drwxr-xr-x - cloudera supergroup          0 2025-06-16 23:40 /user/root/tag
drwxr-xr-x - root    supergroup          0 2025-06-15 02:30 /user/root/vijaya
[cloudera@quickstart ~]$
```

To copy the file **temp.txt** from Local file system location (/user/cloudera/Desktop/temp.txt) use the following command:

6. Cd Desktop // since file is on Desktop

7. hdfs dfs -copyFromLocal temp.txt /user/root/demodir

8. hdfs dfs -ls /user/root/demodir

```
[cloudera@quickstart ~]$ cd Desktop
[cloudera@quickstart Desktop]$ hdfs dfs -copyFromLocal temp.txt /user/root/demodir
[cloudera@quickstart Desktop]$ hdfs dfs -ls /user/root/demodir
Found 1 items
-rw-r--r-- 1 cloudera supergroup          49 2025-06-30 20:47 /user/root/demodir/temp.txt
[cloudera@quickstart Desktop]$
```

To list the content of temp.txt use following command

9. hdfs dfs -cat /user/root/demodir/temp.txt

```
[cloudera@quickstart Desktop]$ hdfs dfs -cat /user/root/demodir/temp.txt
Welcome to the Big Data Session
Cloudera Hadoop

[cloudera@quickstart Desktop]$ █
```

Please Note : Before working with all these commands, HDFS, NameNode, DataNode services must be ON. Open Cloudera Manager and start the services. Command line utility can also be used.

Program 3

Develop a Map Reduce program that mines weather data and displays appropriate messages indicating the weather conditions of the day.

```
import
java.io.IOException;
import java.util.Iterator;
import org.apache.hadoop.fs.Path;
import
org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import
org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import
org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import
org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
import
org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.Job;
i    m    p    o    r    t
org.apache.hadoop.mapreduce.Mapper
r ;                i m p o r t
org.apache.hadoop.mapreduce.Reduce
r ;                i m p o r t
org.apache.hadoop.conf.Configuration
;

public static class MaxTemperatureMapper extends Mapper<LongWritable, Text, Text,Text> {

    @Override

    public void map(LongWritable arg0, Text Value, Context context)

        throws IOException,

        InterruptedException { String line = Value.toString();

        if (!(line.length() == 0))

        {
```

```

String date = line.substring(6, 14);          //date
float temp_Max = Float.parseFloat(line.substring(39, 45).trim()); //maximum temp

float temp_Min = Float.parseFloat(line.substring(47, 53).trim()); //
minimum temp if (temp_Max > 35.0) {

context.write(new Text("Hot Day " + date), new Text(String.valueOf(temp_Max)));

}

if (temp_Min < 10) {

    context.write(new Text("Cold Day " + date),new
        Text(String.valueOf(temp_Min)));} } }

public static class MaxTemperatureReducer extends Reducer<Text, Text, Text, Text> {

public void reduce(Text Key, Iterator<Text> Values, Context

    context) throws IOException,

    InterruptedException {

//putting all the values in temperature variable of type

String String temperature = Values.next().toString();

context.write(Key, new Text(temperature));

}

}

public static void main(String[] args) throws Exception {

    //reads the default configuration of cluster from the configuration xml files
    Configuration conf = new Configuration();
    //Initializing the job with the default configuration of
    the cluster Job job = Job.getInstance(conf);
    job.setJarByClass(MyMaxMin.class);          //Assigning the driver class
    name job.setMapOutputKeyClass(Text.class);  //Key type coming out of
    mapper job.setMapOutputValueClass(Text.class); //value type coming out of
    mapper job.setMapperClass(MaxTemperatureMapper.class); //Defining the
    mapper class name job.setReducerClass(MaxTemperatureReducer.class);
    job.setInputFormatClass(TextInputFormat.class);

    job.setOutputFormatClass(TextOutputFormat.class);
    //Configuring the input path from the filesystem into

```

```

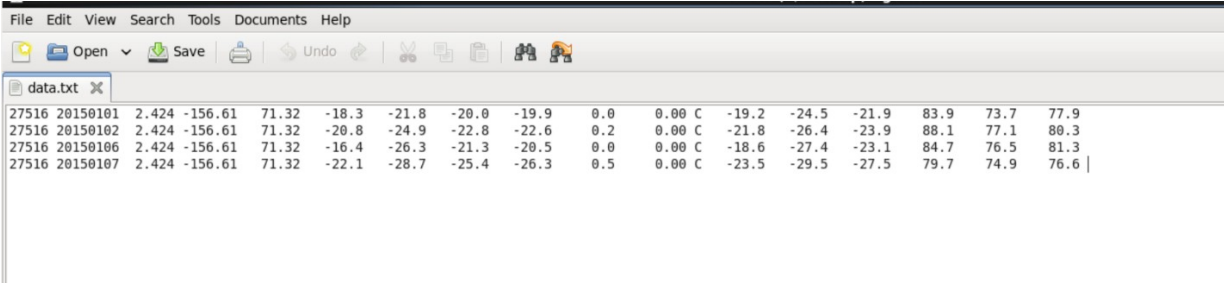
the job Path outputPath = new Path(args[1]);

//Configuring the input path from the filesystem into
the job FileInputFormat. addInputPath(job, new
Path(args[0]));

//Configuring the output path from the filesystem into the job
FileOutputFormat.setOutputPath(job, new Path(args[1]));

job.waitForCompletion(true);
}
}
}
}
INPUT – the input file should be copied to HDFS

```



27516	20150101	2.424	-156.61	71.32	-18.3	-21.8	-20.0	-19.9	0.0	0.00	C	-19.2	-24.5	-21.9	83.9	73.7	77.9
27516	20150102	2.424	-156.61	71.32	-20.8	-24.9	-22.8	-22.6	0.2	0.00	C	-21.8	-26.4	-23.9	88.1	77.1	80.3
27516	20150106	2.424	-156.61	71.32	-16.4	-26.3	-21.3	-20.5	0.0	0.00	C	-18.6	-27.4	-23.1	84.7	76.5	81.3
27516	20150107	2.424	-156.61	71.32	-22.1	-28.7	-25.4	-26.3	0.5	0.00	C	-23.5	-29.5	-27.5	79.7	74.9	76.6

Program 2

Develop a MapReduce program to implement Matrix Multiplication

```
import
java.io.IOException;
import
java.util.HashMap;
import
org.apache.hadoop.conf.
*;
import
org.apache.hadoop.fs.Path;
import
org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import
org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import
org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import
org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import
org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;

public class MatMul {

    public static class MatrixMapper extends Mapper<LongWritable, Text, Text,
        Text> { public void map(LongWritable key, Text value, Context context)
        throws IOException,
            InterruptedException { Configuration conf =
                context.getConfiguration();

                int m = Integer.parseInt(conf.get("m"));

                int p =
                    Integer.parseInt(conf.get("p"));
                String line = value.toString();
```

```

String[] indicesAndValue = line.split(",");
Text outputKey = new Text();           // object for
output key Text outputValue = new Text(); //
object for output value if
(indicesAndValue[0].equals("M"))
{
    for (int k = 0; k < p; k++)
        { outputKey.set(indicesAndValue[1] +
            "," + k);
            outputValue.set("M," + indicesAndValue[2] + "," +
            indicesAndValue[3]); context.write(outputKey, outputValue);
        }
}
else
{
    for (int i = 0; i < m; i++)
    {
        outputKey.set(i + "," + indicesAndValue[2]);
        outputValue.set("N," + indicesAndValue[1] + "," +
            indicesAndValue[3]); context.write(outputKey, outputValue);
    }
}
}

}

public static class MatrixReducer extends Reducer<Text, Text, Text,
    Text> { public void reduce(Text key, Iterable<Text> values,
    Context context)
        throws IOException,
        InterruptedException { String[] value;

    HashMap<Integer, Float> hashA = new HashMap<Integer, Float>();

    HashMap<Integer, Float> hashB = new HashMap<Integer,
    Float>(); for (Text val : values) {
        value = val.toString().split(",");

        if (value[0].equals("M"))

            hashA.put(Integer.parseInt(value[1]),Float.parseFloat(value

```

```

        [2])); else

        hashB.put(Integer.parseInt(value[1]), Float.parseFloat(value[2]));

    }

    int n =

    Integer.parseInt(context.getConfiguration().get("n"));

    float result = 0.0f;

    for (int j = 0; j < n; j++) {

        float a_ij = hashA.containsKey(j) ?
        hashA.get(j) : 0.0f; float b_jk =
        hashB.containsKey(j) ? hashB.get(j) : 0.0f;
        result += a_ij * b_jk;
    }

    if (result != 0.0f)

        context.write(null, new Text(key.toString() + "," + Float.toString(result)));

    }

}

public static void main(String[] args) throws

    Exception { Configuration conf = new

        Configuration();

    // M is an m-by-n matrix; N is an n-by-p
    matrix.conf.set("m", "2");

    conf.set("n", "2");
    conf.set("p", "2");
    Job job = Job.getInstance(conf,
    "MatrixMultiplication");
    job.setJarByClass(MatMul.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);
    job.setMapperClass(MatrixMapper.class);
    job.setReducerClass(MatrixReducer.class);
    job.setInputFormatClass(TextInputFormat.class);
    job.setOutputFormatClass(TextOutputFormat.class)
    ; FileInputFormat.addInputPath(job, new
    Path(args[0]));
    FileOutputFormat.setOutputPath(job, new
    Path(args[1])); job.waitForCompletion(true);

```

```
}  
}
```

Input File

```
[cloudera@quickstart Desktop]$ hdfs dfs -cat /user/root/matin/matrix.txt  
M,0,0,1  
M,0,1,2  
M,1,0,3  
M,1,1,4  
N,0,0,5  
N,0,1,6  
N,1,0,7  
N,1,1,8  
[cloudera@quickstart Desktop]$ █
```

OutputFile

```
Bytes written=36  
[cloudera@quickstart Desktop]$ hdfs dfs -ls /user/root/matout/  
Found 2 items  
-rw-r--r-- 1 cloudera supergroup 0 2025-06-16 01:44 /user/root/matout/_SUCCESS  
-rw-r--r-- 1 cloudera supergroup 36 2025-06-16 01:44 /user/root/matout/part-r-00000  
[cloudera@quickstart Desktop]$ hdfs dfs -ls /user/root/matout/part-r-00000  
-rw-r--r-- 1 cloudera supergroup 36 2025-06-16 01:44 /user/root/matout/part-r-00000  
[cloudera@quickstart Desktop]$ hdfs dfs -cat /user/root/matout/part-r-00000  
0,0,19.0  
0,1,22.0  
1,0,43.0  
1,1,50.0  
[cloudera@quickstart Desktop]$ █
```

Program 4

Develop a MapReduce program to find the tags associated with each movie by analyzing movie lens data.

```
import
java.io.IOException;
import java.util.*;

import
org.apache.hadoop.conf.Configuratio
n; import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.*;
import

org.apache.hadoop.mapreduce.lib.output.FileOutputForma

t; public class MovieTagsJoin {

    public static class MovieMapper extends Mapper<LongWritable, Text, Text, Text> {

        public void map(LongWritable key, Text value, Context
            context) throws IOException, InterruptedException {

            String line = value.toString();
            if (key.get() == 0 && line.contains("movieId")) return; // skip header

            String[] fields = line.split(",", 3); //
            movieId,title,genres if (fields.length >= 2) {

                String movieId =
                    fields[0].trim(); String
                    title = fields[1].trim();
                context.write(new Text(movieId), new Text("MOVIE:." + title));
            }

        }
    }
}
```

```

}
public static class TagMapper extends Mapper<LongWritable, Text, Text,
    Text> { public void map(LongWritable key, Text value, Context
    context)

        throws IOException, InterruptedException {
            String line = value.toString(); // Read the line from the tags.txtif
            (key.get() == 0 && line.contains("userId")) return; // skip header String[]
            fields = line.split(",", 4); // userId,movieId,tag,timestamp
            if (fields.length >= 3) {
                String movieId =
                fields[1].trim(); String tag
                = fields[2].trim();
                context.write(new Text(movieId), new Text("TAG::" + tag));
            }
        }
    }
}

```

```

public static class JoinReducer extends Reducer<Text, Text, Text,
    Text> { public void reduce(Text key, Iterable<Text> values,
    Context context)

        throws IOException,
        InterruptedException { String
        movieTitle = null;
        List<String> tags = new
        ArrayList<>(); for (Text val :
        values) {

            String value = val.toString();
            if
                (value.startsWith("MOV
                IE::")) movieTitle =
                value.substring(7);
            }
        }
    }
}

```

```

        else if
            (value.startsWith("TAG::
            "))
            tags.add(value.substring(
            5));

        }

    if (movieTitle != null && !tags.isEmpty())
        { context.write(new Text(movieTitle), new
        Text(", " + tags));
        }
    }
}

public static void main(String[] args) throws
    Exception { Configuration conf = new
    Configuration();

    Job job = Job.getInstance(conf, "Movie Tags Join");
    job.setJarByClass(MovieTagsJoin.class);

    // Set Mappers
    MultipleInputs.addInputPath(job, new Path(args[0]), TextInputFormat.class,
    MovieMapper.class); MultipleInputs.addInputPath(job, new Path(args[1]),
    TextInputFormat.class, TagMapper.class);

    job.setReducerClass(JoinReducer.class);

    // Output Types

    job.setOutputKeyClass(Text.cl
    ass);

    job.setOutputValueClass(Text.c
    lass);

    FileOutputFormat.setOutputPath(job, new
    Path(args[2])); job.waitForCompletion(true);
}

```

}

INPUT Files on the Desktop – 2 Files 1.Movie.txt 2. tags.txt

```
[cloudera@quickstart ~]$ hdfs dfs -cat /user/root/movie/Movie.txt
movieId title genres
1,Toy Story (1995),Adventure|Animation|Children|Comedy|Fantasy
2,Jumanji (1995),Adventure|Children|Fantasy
3,rumpier Old Men (1995),Comedy|Romance

[cloudera@quickstart ~]$ hdfs dfs -ls /user/root/tag
Found 1 items
-rw-r--r-- 1 cloudera supergroup 122 2025-06-16 23:40 /user/root/tag/Tags.txt
[cloudera@quickstart ~]$ hdfs dfs -cat /user/root/tag/Tags.txt
userId movieId tag timestamp
15,1,funny,1139045764
15,2,childish,1139045874
20,1,pixar,1139045984
20,3,oldie,1139046064
```

OUTPUT

```
Bytes Written=95
[cloudera@quickstart Desktop]$ hdfs dfs -cat /user/root/movietags1/part-r-00000
Toy Story (1995) , [pixar, funny]
Jumanji (1995) , [childish]
rumpier Old Men (1995) , [oldie]
[cloudera@quickstart Desktop]$ █
```

Program 6

Write Pig Latin scripts to sort, group, join, project, and filter the data.

```
-load data
students = LOAD '/user/root/pigdata/students.txt' USING
PigStorage(',') AS (id:int, name:chararray, dept:chararray,
marks:int);

-- Filter students with marks > 80
high_scorers = FILTER students BY marks > 80;

-- Sort all students by marks descending
sorted_students = ORDER students BY marks
DESC;

--Project only name and marks
projected = FOREACH students GENERATE name, marks;

-- Group by department and find average
marks grouped = GROUP students BY dept;
average_marks = FOREACH grouped GENERATE group AS department, AVG(students.marks)
AS avg_marks;

STORE sorted_students INTO '/user/root/pigoutput/sorted_students' USING
PigStorage(','); STORE high_scorers INTO '/user/root/pigoutput/high_scorers'
USING PigStorage(','); STORE projected INTO '/user/root/pigoutput/projected'
USING PigStorage(',');
STORE average_marks INTO '/user/root/pigoutput/average_marks' USING PigStorage(',');
```

Procedure:

1. Open the new document on the desktop of cloudera Hadoop. Copy and Paste the content. – indicates the comment line. The file is saved for example in **/home/cloudera/workspace/PigExample.pig** **Students.txt** is the input file(uploaded in the drive).
2. Copy the input file to HDFS.
3. Navigate to the location where pig script file is written and Run the script with following command. Output file location is given in the script itself.

Pig -x mapreduce PigRexample.pig /user/root/pigdata/studets.txt

INPUT FILE

```
[cloudera@quickstart workspace]$ hdfs dfs -cat /user/root/pigdata/students.txt
101,John,CS,85
102,Alice,IT,92
103,Bob,CS,76
104,David,EC,89
105,Eve,IT,67
106,john,AI ML,88
107,George,EC,100
```

OUTPUT FILE : one of the output file is shown with part-m-00000 in the dir projected.

```
[cloudera@quickstart workspace]$ hdfs dfs -ls /user/root/pigoutput/
Found 4 items
drwxr-xr-x - cloudera supergroup          0 2025-06-16 03:06 /user/root/pigoutput/average_marks
drwxr-xr-x - cloudera supergroup          0 2025-06-16 03:06 /user/root/pigoutput/high_scorers
drwxr-xr-x - cloudera supergroup          0 2025-06-16 03:06 /user/root/pigoutput/projected
drwxr-xr-x - cloudera supergroup          0 2025-06-16 03:07 /user/root/pigoutput/sorted_students
[cloudera@quickstart workspace]$ hdfs dfs -ls /user/root/pigoutput/projected
Found 2 items
-rw-r--r-- 1 cloudera supergroup          0 2025-06-16 03:06 /user/root/pigoutput/projected/_SUCCESS
-rw-r--r-- 1 cloudera supergroup        61 2025-06-16 03:06 /user/root/pigoutput/projected/part-m-00000
[cloudera@quickstart workspace]$ hdfs dfs -cat /user/root/pigoutput/projected/part-m-00000
John,85
Alice,92
Bob,76
David,89
Eve,67
john,88
George,100
```

Program 8

Implement a word count program in Hadoop and Spark.

Please Note: Open colab and copy paste the code. The input file is to be uploaded into drive and change the location of the file accordingly in the below code.

File name is word.txt uploaded in the drive. This is spark program for wordcount using python.

```
from pyspark.sql import
SparkSession spark =
SparkSession.builder\
    .master("local")\
    .appName('word_count')\
    .getOrCreate()

from pyspark import
SparkContext sc =
SparkContext.getOrCreate(
)
# Read the input file and Calculating words
count text_file = sc.textFile("/content/
sample_data/word.txt") text_file.collect()

counts = text_file.flatMap(lambda line: line.split(" ")) \
    .map(lambda word: (word, 1)) \
    .reduceByKey(lambda x, y: x
+ y) output = counts.collect()
output
```

Input File content

```
['My name is Vijayalaxmi',  
 'Vijayalaxmi likes to code',  
 'she is working in citech']
```

OUTPUT

```
✓ [('My', 1),  
   ('name', 1),  
   ('is', 2),  
   ('Vijayalaxmi', 2),  
   ('likes', 1),  
   ('to', 1),  
   ('code', 1),  
   ('she', 1),  
   ('working', 1),  
   ('in', 1),  
   ('citech', 1)]
```

Program 7

Use Hive to create, alter, and drop databases, tables, views, functions, and indexes.

1. To start with hive Use hive CLI

hive

```
[cloudera@quickstart ~]$ hive
Logging initialized using configuration in jar:file:/usr/lib/hive/lib/hive-common-1.1.0-cdh5.12.0.jar!/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive> █
```

2. Create Database using following command at hive prompt

Hive> create database organization;

```
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive> create database organization;
OK
Time taken: 65.271 seconds
hive> █
```

3. To show the databases

Hive> show databases;

```
hive> show databases;
OK
default
organization
Time taken: 1.615 seconds, Fetched: 2 row(s)
hive> █
```

4. To create table into organization database, we must use command and create table by giving the schema of the table to be created.

```
hive> use organization;
OK
Time taken: 1.012 seconds
hive> create table employee(
  > id int,
  > name string,
  > salart float)
  > row format delimited
  > fields terminated by ',';
OK
Time taken: 1.568 seconds
hive> █
```

5. To list the tables

Hive> show tables;

```
hive> show tables;
OK
employee
Time taken: 0.193 seconds, Fetched: 1 row(s)
hive> █
```

6. load the data from the file. Ex – emp.txt in the HDFS. The screen shot shows the file being copied to HDFS and listing of the file with fields delimited by comma. (delimiter can be anything)

```
[cloudera@quickstart Desktop]$ hdfs dfs -ls /user/root/
Found 1 items
-rw-r--r--  1 cloudera supergroup      73 2025-07-02 01:56 /user/root/emp.txt
[cloudera@quickstart Desktop]$ hdfs dfs -cat /user/root/emp.txt
100,vijayalaxmi,1000.0
200,vijaya,2000.4
300,jaya,400.3
400,laxmi,5000.0
[cloudera@quickstart Desktop]$ █
```

7. Load data using following load command

Hive> Load data inpath '/user/root/emp.txt' overwrite into table employee;

```
hive> load data inpath '/user/root/emp.txt' overwrite into table employee;
Loading data to table organization.employee
Table organization.employee stats: [numFiles=1, numRows=0, totalSize=73, rawDataSize=0]
OK
Time taken: 3.116 seconds
hive> █
```

8. Query the data using select query

>select * from employee;

```
hive> select * from employee;
OK
100    vijayalaxmi    1000.0
200    vijaya    2000.4
300    jaya    400.3
400    laxmi    5000.0
Time taken: 0.13 seconds, Fetched: 4 row(s)
hive> █
```